

SOFTWARE ENGINEERING FINAL PROJECT: "MESOS"

Group: 32

Components: Grillo Giuseppe Salvatore Pasquale, Hu Xiayi, Iavarone Mattia, Luzzi Denise

Work Division:

- **Model & Controller:** All members
- **RMI:** Luzzi, Hu
- **Socket:** Grillo
- **TUI:** Grillo
- **GUI:** Iavarone
- **Database:** Grillo

1. Architectural Overview & Network Structure

The project delivers a Java-based digital adaptation of the board game "**Mesos**" utilizing the **Model-View-Controller (MVC)** design pattern. The application features a **client-driven** architecture where the server remains passive, processing incoming requests from clients and broadcasting state updates back to them.

Network Framework & Initialization

To bridge user interfaces and the game core, the system utilizes specialized network layers: ClientController handles client-side networking, while ServerController handles server-side traffic.

- **Server Startup:** The Server class instantiates global components (GameManager, Notifier, ServerController, and Lobby), binds both **RMI** and **Socket** protocols to the same ServerController instance so that Clients can connect using either protocol.
- **Pre-game Lobby Management:** A Lobby component manages user authentication before the match starts. Unverified connections are tracked as TempPlayer objects holding a client reference, a temporary username, and a selected totem. Upon validation, they become official Player instances. If the first player restricts the match size below the number of active connections, excess clients are disconnected via checkMoreThenEnoughPlayers(). Any modifications inside the lobby trigger immediate updates through the ServerController.
- **Client Implementation:** ClientMain prompts the user for the server IP, builds the ClientController, and starts either the **GUI** (centered around GuiView) or the **TUI**.
- The client holds a copy of the game board that gets updated via network for every state modify or replaced at the start of each round.

2. Card Ingestion & Data Management (File Parsing)

Game content is externalized into versioned JSON source files within the Resources directory, segmented by historical Eras and distributed across three primary gameplay typologies: Characters, Buildings, and Events.

Data Transport & Factory Pipeline

The initialization system splits raw storage properties from active object mechanics by using Data Transfer Objects (DTO):

- **JSON Parsing:** The Loader class uses the **Gson** library to map raw text to distinct DTO objects. The loadFile(era) method extracts data into an EraDTO, while loadBuildingsByEra() creates a structured building registry.

- **Domain Model Synthesis:** Domain translation is handled by specialized factories. The BuildingFactory reads structural records to output functional objects inside the Buildings package. Concurrently, ConcreteFactoryEra processes EraDTO inputs to generate character and event cards inside the Cards package.

3. Implemented Design Patterns

Abstract Factory Pattern

Used to cleanly decouple game card creation from JSON serialization details. The application relies on ConcreteFactoryEra and BuildingFactory to translate raw DTO properties into runtime domain objects without exposing generation rules directly to the gameplay controllers.

Visitor Pattern

Applied across multiple game cards to handle polymorphic mechanics without changing existing classes:

- **Buildings:** We implemented several visitor patterns to distinguish the various building effects and activate them when necessary, taking full advantage of Java's polymorphism.
- **Characters:** Characters implement an addCard method that accepts a CharacterVisitor. Specialized visitors (such as AddAndCountCharacter) then identify the precise subclass and automatically update the player's internal resource counters.

Command Pattern

Utilized exclusively to structure text parsing within the TUI and the Socket communication protocol.

- **Decoupling Input from Logic:** Instead of processing terminal commands through long conditional blocks, the TUI converts text entries (e.g., login actions, totem reservations, token placement, card acquisitions, turn skips) into discrete executable command instances.
- **Execution Flow:** Each individual command handles its own parameter validation and communicates directly with the ClientController. This structure isolates string parsing away from the core networking layer, making the TUI easily extendable when adding new game operation

Observer Pattern

This pattern establishes the primary broadcast infrastructure between the server and active game sessions via the `Notifier` component. Upon successful connection, each client is immediately registered into a central subscriber registry. The `Notifier` utilizes this registry to broadcast real-time state changes or issue immediate validation errors whenever a client attempts an out-of-turn or invalid gameplay action.

Test Percentages

```
> 📁 Controller 100% classes, 89% lines covered
> 📁 Database 100% classes, 86% lines covered
> 📁 FileLoader 75% classes, 67% lines covered
> 📁 Model 95% classes, 92% lines covered
```